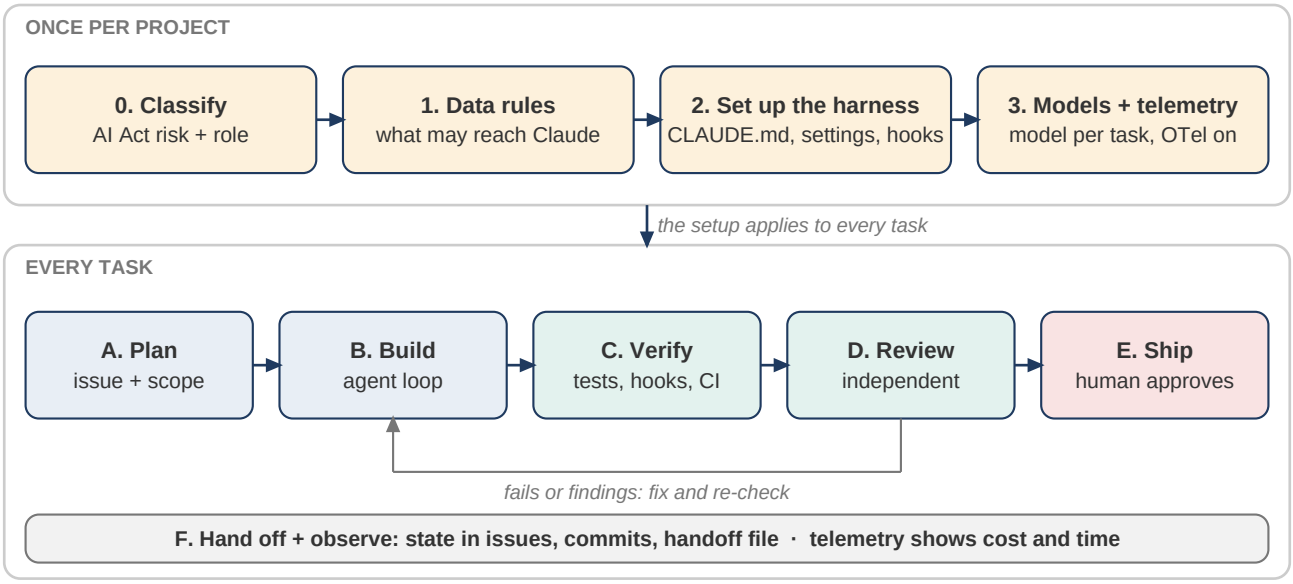
A recommended workflow with Claude

How to set up and run work with Claude so it is efficient, safe, compliant, low on tokens and predictable

A good workflow follows one rule above the others: **the model proposes, the harness decides**. Claude is fast and capable but probabilistic, billed per token and able to act on your files and systems. You get the benefits without the risks by deciding a few things once per project, by enforcing the important rules in code rather than in chat, and by running every task through the same plan → build → verify → review → ship cycle.

The overall lifecycle



Feed what you learn back into CLAUDE.md, hooks and model choice (step F → steps 2–3)

Figure 1. Setup happens once and is revisited when the purpose of the system changes. Every task then runs through the same cycle, and what is learned feeds back into the setup.

Part 1: Set up once per project

These four steps take an hour or two at the start and save far more later. Write each decision down in the repo, even when the answer is "minimal risk" or "defaults are fine".

Step 0: Classify the use case (EU AI Act)

- Describe the intended purpose in one sentence: who uses the system and which decisions it affects.
- Check against the prohibited practices (Art. 5) and the high-risk areas (Annex III), such as recruitment, credit scoring and education including assessment of students.
- Decide your role for each system: provider, deployer or both. Rebranding, substantially modifying or repurposing a system can make you the provider.
- Using Claude as a development tool is normally minimal risk. What you *build* with it may not be.

Step 1: Decide data rules

- What may be sent to Claude: no real personal data, secrets or customer data unless explicitly allowed and covered by agreements and GDPR.

- What may be logged: keep telemetry at metadata only (the default).
- Turn each rule that must never be broken into a check in code, for example a script that blocks personal identity numbers before commit.

Step 2: Set up the harness

- `CLAUDE.md` in the repo with goals, decisions, conventions and hard rules. Keep it short and concrete; point to longer decision files.
- `.claude/settings.json` with permissions: reading and tests on allow, push and PRs on ask, merge, force push and secrets on deny.
- Hooks for rules that must always run (PreToolUse for checks, PostToolUse for formatting and linting).
- Skills for recurring tasks so they are done the same way every time.
- External safeguards that apply even without Claude: a git pre-commit hook, CI tests and a branch ruleset that protects `main`.

Step 3: Plan models and turn on telemetry

- Decide a default model per task type (see Figure 4) and set cheaper models for sub-agents that only search or summarise.
- Turn on OpenTelemetry so you can see cost per model, failing tools and where time goes. Keep prompt and response content off.

Where each part lives

Location	Example	Shared with the team
Your machine, outside the repo	`~/.claude/settings.json`, telemetry variables, memory, `gh` login	No
Repo, checked in	`CLAUDE.md`, `.claude/settings.json`, `.claude/agents/`, check scripts, CI config	Yes
Repo, git-ignored	`handoff.local.md`, `.env`	No
GitHub	issues, PRs, review comments, branch ruleset	Yes
Claude project	decision documents and project instructions	Yes

Rule of thumb: what should apply to everyone working in the repo is checked in; what applies to one person or one machine stays outside.

The harness in layers

Context above the model steers what it knows. Guardrails below it steer what actually happens. External safeguards still apply when someone works without Claude, so an important rule is enforced in more than one layer.

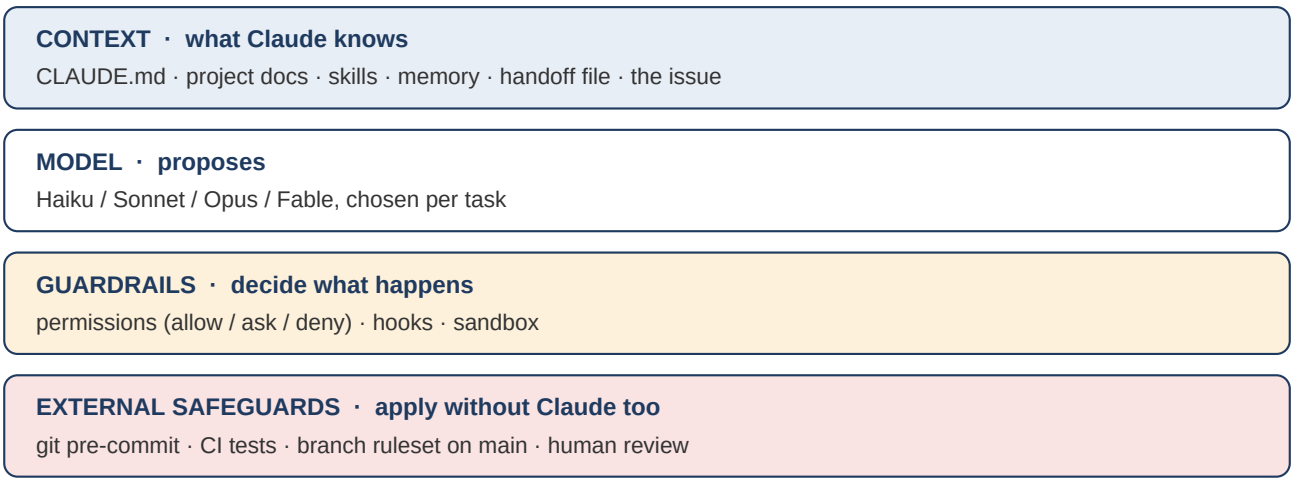


Figure 2. The four layers of a Claude workflow.

Part 2: Run every task the same way

A. Plan

- Start from an issue with a clear goal and acceptance criteria. The issue, not the chat, is where the task lives.
- For anything non-trivial, ask Claude for a plan first and approve it before any code is written.
- Pick the model for the task. Split large tasks so each part can use the cheapest model that can do it.

B. Build: the agent loop

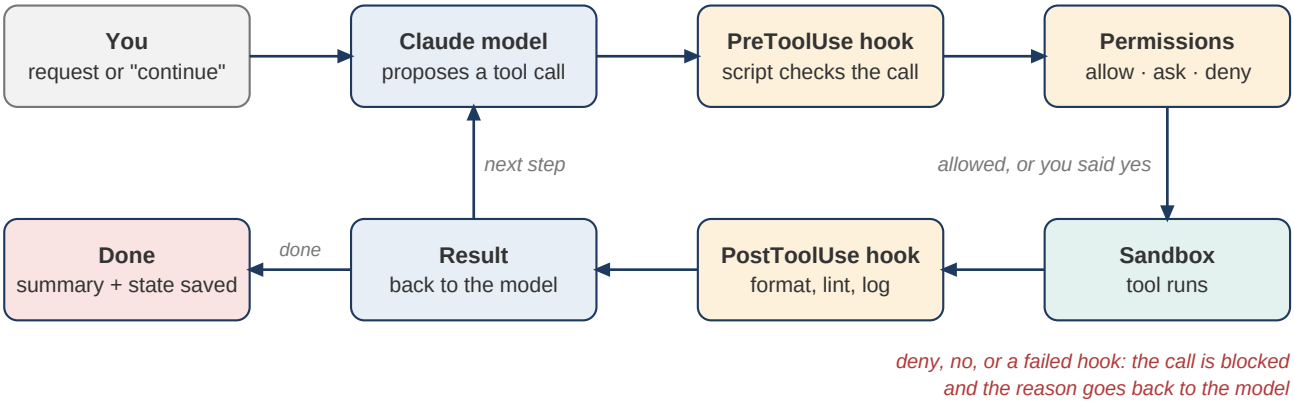


Figure 3. Claude never runs anything itself. Every call passes hooks, permissions and the sandbox, and the result goes back to the model until the task is done.

C. Verify

- Let tests, type checks and linters decide whether the work is correct, not Claude's own judgement.
- Anything the system depends on should follow a fixed format (for example a JSON schema), and be validated.
- Exact logic (calculations, rules, permissions, personal data) lives in ordinary code that Claude writes and tests, not in model output at runtime.

D. Review independently

- Before merge, run a review with a strong model that has not seen the builder's reasoning, only the diff and the requirements.
- Verify each finding before fixing it; a reviewer can also be wrong.
- This is the step where a missed error costs the most, so do not save tokens here.

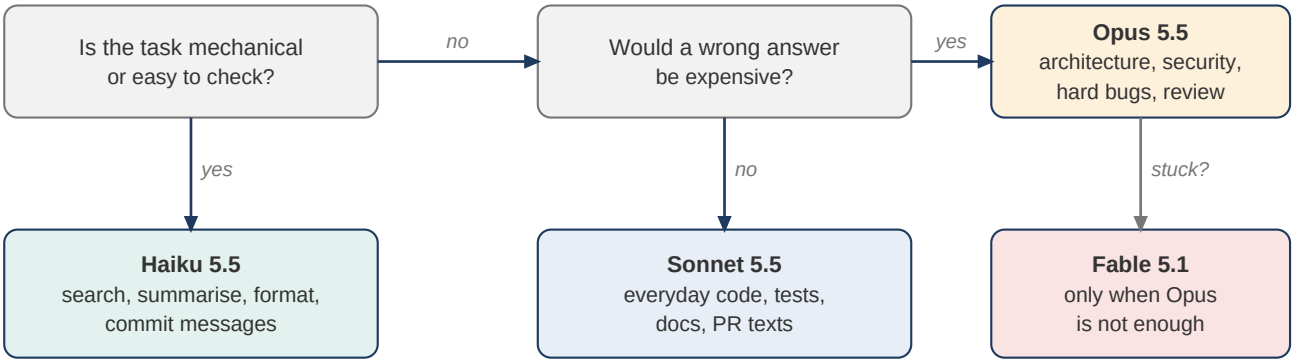
E. Ship with a human in charge

- Push, PRs and public comments are on ask; merge to `main` is done by a person and protected by the ruleset.
- Store AI output that must be reproducible or auditable, since asking again does not guarantee the same answer.

F. Hand off and observe

- Move state out of the chat: issue comments, PR descriptions, commits and a git-ignored handoff file let the next session start with "continue".
- Review telemetry regularly: is model choice saving what you expected, which tools fail, how often are permissions denied?
- When Claude makes the same mistake twice, fix the setup (CLAUDE.md, a hook, a skill), not just the output.

Choosing the model



Escalate when a smaller model goes in circles; step down again when work becomes routine.

Figure 4. Choose the cheapest model that can do the task. Output prices differ roughly 100× between Haiku 5.5 and Fable 5.1, and most tokens go to reading and searching, which need little reasoning.

Part 3: Starter configuration

These are minimal starting points to adapt. Check exact syntax against the current Claude Code documentation.

CLAUDE.md

```
# Project: <name>
## Purpose
One sentence: who uses it and what decisions it affects.
AI Act: <risk level>, our role: <provider/deployer>. See docs/ai-act.md

## Decisions (read these first)
- docs/decisions.md   - dated decisions, newest first

## Hard rules (also enforced by hooks and CI)
- No real personal data in code, tests, prompts or logs
- Never commit .env or secrets
- All changes go through a PR; never push to main

## Conventions
- Run `make test` before saying a task is done
- Keep changes small; one issue per PR
- Update handoff.local.md at the end of each session
```

.claude/settings.json

```
{
  "permissions": {
    "allow": ["Bash(make test:*)", "Bash(git status)", "Bash(git diff:*)"],
    "ask":   ["Bash(git push:*)", "Bash(gh pr create:*)", "Bash(gh issue comment:*)"],
    "deny":  ["Bash(git push --force:*)", "Bash(gh pr merge:*)", "Read(./.env)"]
  },
  "hooks": {
    "PreToolUse": [{ "matcher": "Bash",
      "hooks": [{ "type": "command",
                  "command": "python3 scripts/check_personal_data.py" }] }]
  }
}
```

Telemetry (in your shell or ~/.claude/settings.json, not the repo)

```
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=otlp
export OTEL_LOGS_EXPORTER=otlp
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
# Prompt, response and tool content stay redacted by default.
```

A cheap sub-agent (.claude/agents/searcher.md)

```
---
name: searcher
description: Finds files and summarises code. Read-only.
model: haiku
tools: Read, Grep, Glob
---
Search the codebase and return file paths with a short summary.
```

Checklist

Area	Done when
Compliance	Purpose, AI Act risk level and role are written down; prohibited practices ruled out; AI literacy measures documented
Data	Rules for what may reach Claude and the logs are written; the most important rule is checked in code
Context	`CLAUDE.md` exists, is short, and points to dated decision files
Guardrails	Permissions set (allow / ask / deny); hooks for hard rules; sandbox on
External safeguards	Pre-commit hook, CI tests and a ruleset protecting `main`
Models	Default model per task type; cheap sub-agents for search; strong model for review
Determinism	Exact logic in code; fixed output formats validated; model version pinned in API use; important AI output stored
Observability	Telemetry on with content redacted; someone looks at it regularly
Task cycle	Every task: issue → plan → build → verify → independent review → human merge → handoff
Improvement	Repeated mistakes lead to changes in CLAUDE.md, hooks or skills

Common pitfalls

- Rules that only exist in a chat. They are gone in the next session; put them in CLAUDE.md or code.
- Relying on instructions for things that must never happen. Instructions can be missed; hooks and deny rules cannot.
- Running everything on the biggest model. It costs several times more without making routine work better.
- A model that is too small for the task. Several failed attempts cost more than one escalation.
- Switching model mid-task without reason, which forces context to be rebuilt.
- Turning on content logging "just to debug" and forgetting it.
- Discovering a high-risk use case after launch instead of classifying it on day one.

Prices, product features and regulatory dates change. Verify against current documentation and the consolidated text of Regulation (EU) 2024/1689 as amended by (EU) 2026/1744. This document is guidance, not legal advice.